

UNIT-01

ALGORITHM DESIGNING

ALGORITHM - An Algorithm is a well defined sequential process or set of rules to be followed in calculations and other problem solving operations that accepts a value or collection of values as input and produces the output(s).

An Algorithm is independent of the language used. It tells the programmer the logic used to solve the problem.

Why do we need Algorithms?

- In terms of designing a solution to an IT problem. Computers are fast but not infinitely fast. The memory may be inexpensive but not free, so, computing time is therefore a bounded resource and so is the space in memory. So we should use these resources wisely and here come the algorithms that are efficient in terms of time and space will help you do so.

Characteristics of an algorithm -

- The algorithm should be clear and unambiguous
- There should be 0 or well defined inputs to an algorithm
- An algorithm must produce one or more well-defined outputs that are equivalent to the desired
- Algorithms must stop or end after a finite number of steps.

What is the Analysis of an algorithm?

→ Analysis of an algorithm is the process of finding the Computational Complexity of any algorithm. By Computational Complexity, I'm referring to the amount of time taken, space and any other resources needed to execute the algorithm.

The goal of algorithm analysis is to compare different algorithms that are used to solve the same problem. This is done to evaluate which method solves a problem more quickly, efficiently and memory wise.

To analyze an algorithm we use two factors namely 'Time Complexity' and 'Space Complexity'.

1. Time Complexity - Time Complexity is the Computational Complexity that describes the amount of time taken by an algorithm to execute. Time Complexity is commonly estimated by counting the number of elementary operations performed by the algorithm such as iterative statements, loops and comparisons.

2. Space Complexity - Space Complexity of an algorithm is the ~~time~~ space or the total amount of memory required by an algorithm to complete the execution.

ASYMPTOTIC NOTATIONS

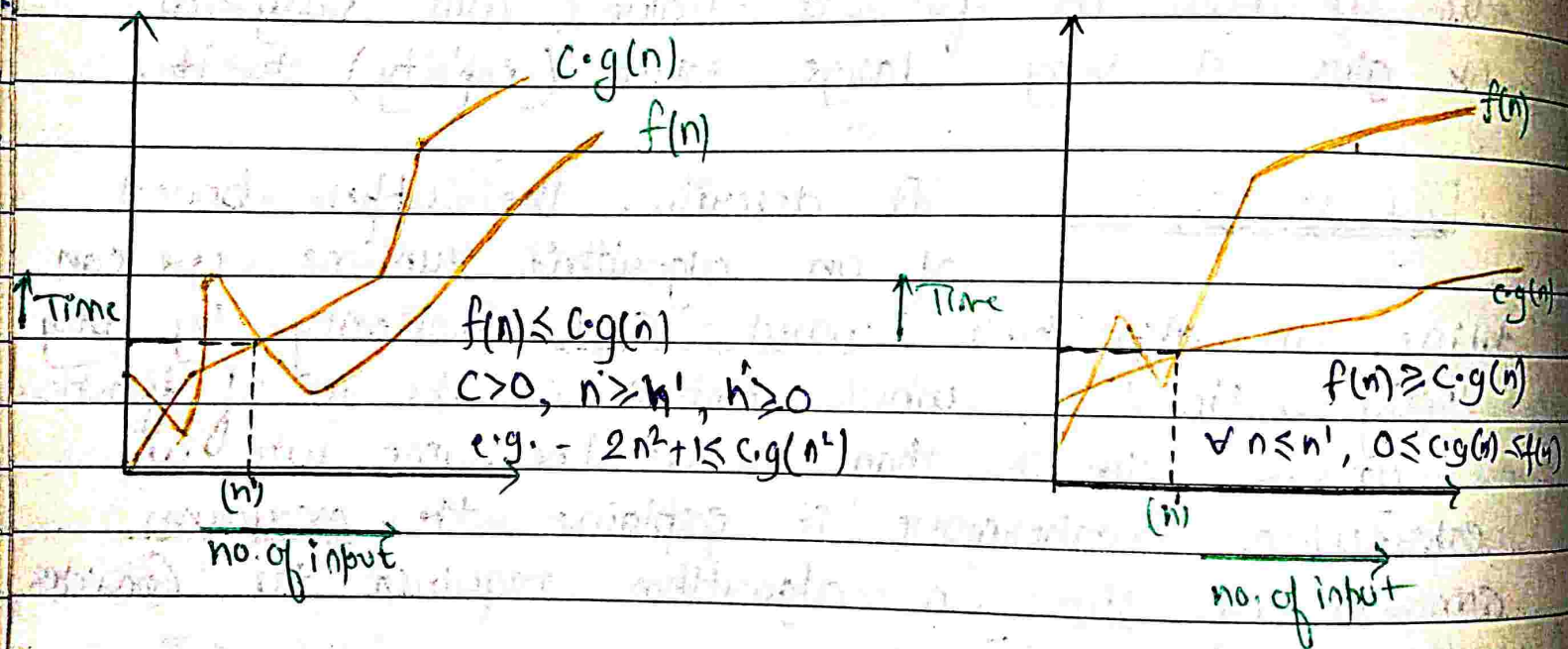
Asymptotic Notations are mathematical Notations that are used to analyse the runtime of a given algorithm for a large input. It helps us to compare the runtime of different algorithms without actually calculating their runtime manually. They are used only for large inputs.

For an asymptotic function, the asymptotic behaviour tells us how the function's value will vary as we give a very large input (infinity) to it.

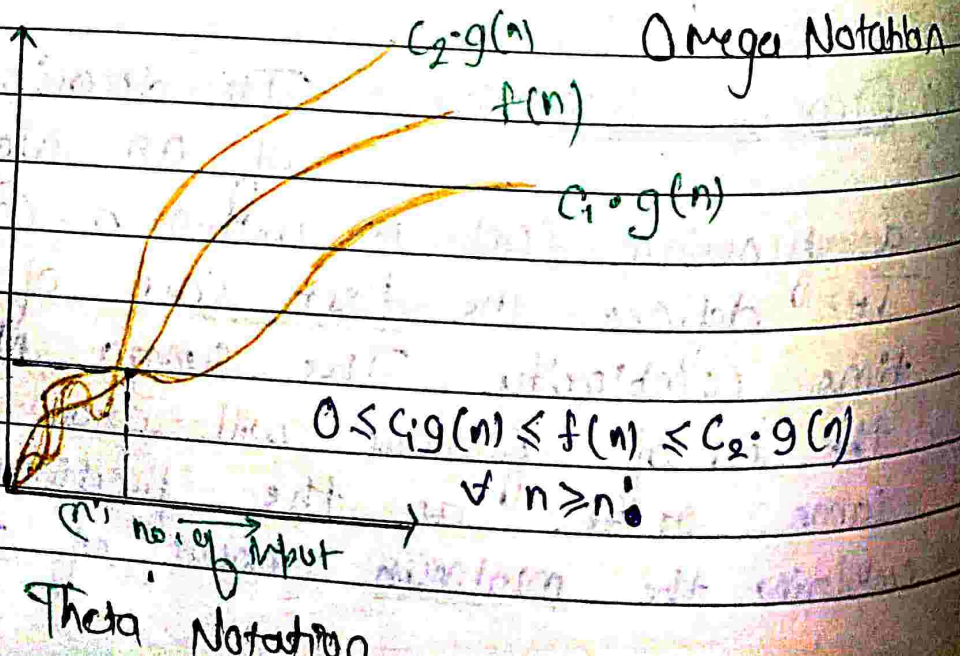
1. Big-O Notation - It describes the upper bound of an algorithm's runtime. We can define an algorithm's worst-case Complexity by using Big-O Notation, which determines the set of functions that grow slower than or at the same rate as the expression. Furthermore, it explains the maximum amount of time an algorithm requires to consider all input values.

2. Omega Notation - It describes the lower bound of an algorithm's runtime, for a function $f(n)$ to within a constant factor 'c'. It defines the best case of an algorithm's time complexity. The Omega Notation defines whether the set of functions will grow faster or at the same rate as the functions. Furthermore, it explains the minimum amount of time an algorithm requires.

3. Theta Notation - It describes the middle bound for a function $f(n)$ to within a constant factor 'c'. It defines the average case of an algorithm's time complexity. Theta notation defines when the set of functions lies in both O -expression and Ω -expression, then Theta notation is used.



Big O-Notation



Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Selection	$\sim n(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Bubble	$\sim n(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Insertion	$\sim n(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Heap	$\sim n(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(1)$
Quick	$\sim n(n \log n)$	$\Theta(n \log n)$	$O(n^2)$	$O(n)$
Merge	$\sim n(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(n)$
Linear search	$O(N)$	$O(N)$	$O(N)$	
Binary search	$O(1)$	$O(\log n)$	$O(\log n)$	

HEAP SORT

Heapsort is one of the sorting algorithms used to arrange a list of elements in order. This algorithm uses one of the tree concept called Heap tree and it is an efficient comparison based sorting algorithm that creates a heap from the input array by taking advantage of a heap's properties. In this sorting algorithm, we use max heap to sort array in ascending order and min heap to sort the array in descending order.

Algorithm - step-1 : Build a heap from the input array.
 Build a max heap to sort in ascending order and min heap to sort in descending order.

- Step 2: Swap the root element with the last item of heap
- Step 3: Reduce the size of the heap by one
- Step 4: Heapify the remaining elements of the new sized heap
- Step 5: Repeat steps 2, 3 and 4 as long as the size of the heap is greater than two.
- Step 6: Compare the last two remaining nodes and insert the appropriate element to the array {sorted}.

Pseudocode - $\text{heapify}(\text{array}, n, \text{index})$

↑

size = array.size();

largest = index;

left = $2 * \text{index} + 1$;

right = $2 * \text{index} + 2$;

if (left < n && array[left] > array[largest])
{

largest = left;

}

if (right < n && array[right] > array[largest])
{

largest = right;

if (largest != index)

{

swap(array[index], array[largest])

heapify(array, n, largest)

}

}

DIVIDE & CONQUER

Page: _____
Date: ____/____/____

This Paradigm, breaks a problem into subproblems that are similar to the original problem, recursively solves the subproblems, and finally combines the solutions to the subproblems to solve the original problem. Because divide-and-conquer solves subproblems recursively, each subproblem must be smaller than the original problem. This method has three parts -

1. Divide - divides the problem into a number of subproblems that are smaller instances of the same problem.
2. Conquer - Conquer the subproblems by recursively calling them.
3. Combine - Combine the solutions to the subproblems into the solution for the original problem.

BINARY SEARCH

Binary Search is a fast searching algorithm. This search algorithm works on the principle of Divide & Conquer. For this algorithm to work properly, the data collection should be in the sorted form.

- Algorithm -
- Step 1: Begin with the mid element of the whole array as a search key
 - Step 2: If the value of the search key is equal to the item then return an index of the item
 - Step 3: Or if the value of the search key is less than

the item in the middle of the array, narrow the interval to the lower half.

step 4: else narrow it to the upper half.

step 5: repeatedly check for upper or narrow half as same until the value is found or the interval is empty.

Pseudo Code - Procedure Binary-Search

$A \leftarrow$ sorted Array

$n \leftarrow$ size of Array

$x \leftarrow$ value to be searched

set lowerBound = 1

set upperBound = n

While x not found

if (upperBound < lowerBound)

Exit: x does not exist

set midpoint = lowerBound + (upperBound - lowerBound) / 2

if $A[\text{midpoint}] < x$

set lowerBound = midpoint + 1

if $A[\text{midpoint}] > x$

set upperBound = midpoint - 1

if $A[\text{midpoint}] = x$

Exit: x found at location midpoint

end while

end Procedure

e.g - To understand binary search let us take an sorted Array {10, 14, 19, 26, 27, 31, 33, 35, 42, 44} let us assume that we have to find the location of Value 31 using binary search.

Step-1: We shall determine the middle value of the array by using below formula -

$$\text{mid} = \text{lb} + (\text{ub} - \text{lb}) / 2$$

Here it is, $0 + (9 - 0) / 2 = 4$ {abs(4-5) = 4}

0	1	2	3	4	5	6	7	8	9
10	14	19	26	27	31	33	35	42	44
↓ low				↓ mid					↓ high

Step-2: Now we compare the value of the mid with the value being searched i.e. 31. We find that $31 > 27$, as we have a sorted array then we also know that the target value must be in upper half. So now we'll search in the upper half of the original array, and our new array will be.

5	6	7	8	9
31	33	35	42	44

Step-3: We'll change our low to mid+1 and find a new value of mid again.

$$\text{low} = \text{mid} + 1 = 4 + 1 = 5$$

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2 = 5 + (9 - 5) / 2 = 7$$

5	6	7	8	9
31	33	35	42	44
↓ low		↓ mid		↓ high

Step - 4 - Now we will compare the value of mid with the value being searched, we find that $31 < 35$, and we have a sorted array so the target value must be in the lower portion of the array. so our new array will be.

5	6
31	33
↓	↓
low	mid/high

Step - 5 - Now we are left with two elements in the array, so we'll compare both these elements. On comparing them we find that $31 < 33$, so we'll search in the left side. And we found that 31 is at fifth index.

MERGE SORT

Merge sort is a sorting algorithm based on the divide and conquer strategy. It works by recursively dividing the array into two halves, then sort them and combine them. It takes a time of $n(\log n)$ in the worst case.

Algorithm - step-1: Find the middle index of the array, $\text{middle} = 1 + (\text{last} - \text{first}) / 2$

step-2: Divide the two halves from the middle of the array.

step-3: Call mergesort () for the first half of the array.

step-4: Call mergesort () for the second half of the array.

step-5: Merge the two sorted halves into a single sorted array.

Pseudo Code - SUBROUTINE mergeSort (array)

If $\text{len}(\text{array}) > 1$ Then

$\text{halfArray} = \text{len}(\text{array}) / 2$

$\text{FirstHalf} = \text{array} [: \text{halfArray}]$

$\text{SecondHalf} = \text{array} [\text{halfArray} :]$

mergeSort (FirstHalf)

mergeSort (SecondHalf)

$k = 0$

while $i < \text{len}(\text{FirstHalf})$ and $j < \text{len}(\text{SecondHalf})$

If $\text{FirstHalf}[i] < \text{SecondHalf}[j]$ Then

$\text{array}[k] = \text{FirstHalf}[i]$

$i = i + 1$

Else

$\text{array}[k] = \text{SecondHalf}[j]$

$j = j + 1$

$k+1$

EndIF

Endwhile

EndIF

Outputlist (array)

ENDSUBROUTINE

SUBROUTINE Output list ()

FOR i in len(array)

OUTPUT array[i]

ENDFOR

ENDSUBROUTINE

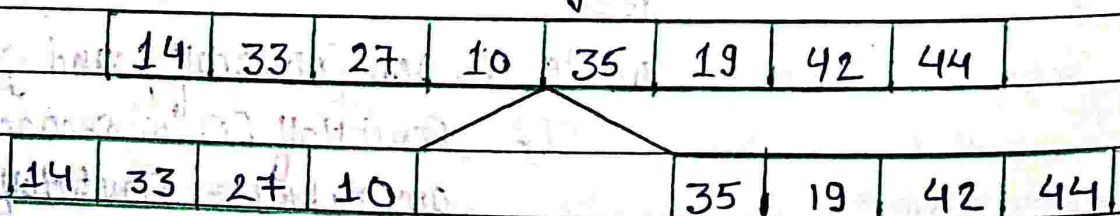
array = []

mergesort (array)

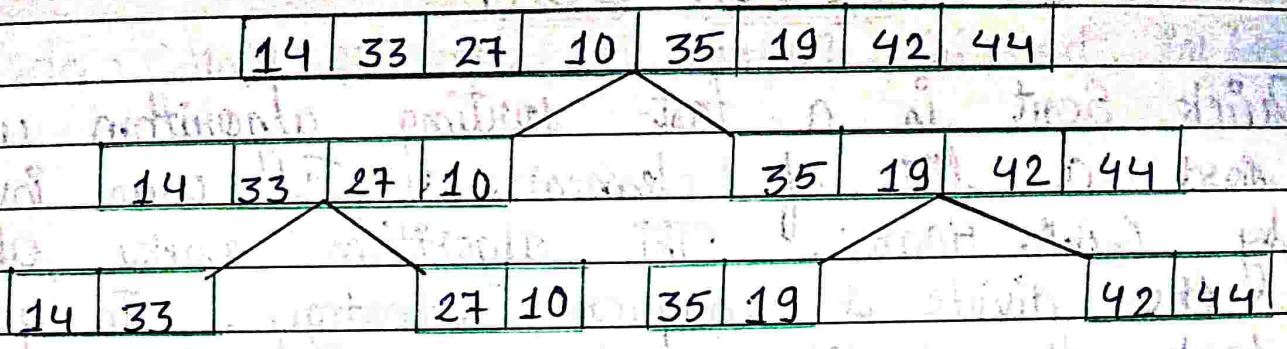
e.g. - To understand merge sort let us take an unsorted array - {14, 33, 27, 10, 35, 19, 42, 44} and trying sort it using merge sort.

Step 1 - Divide the array into two halves using
 $halfArray = len(array) / 2$

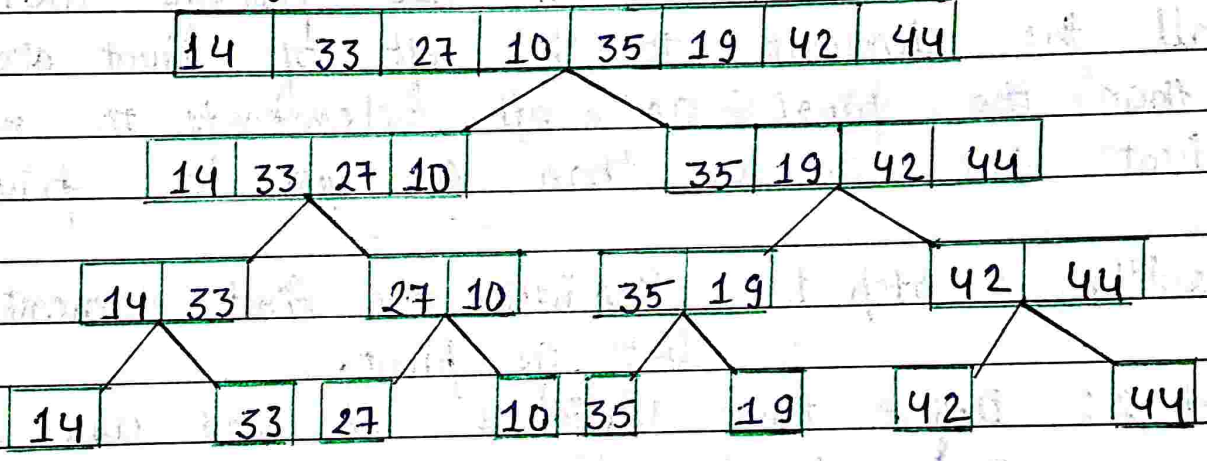
So,



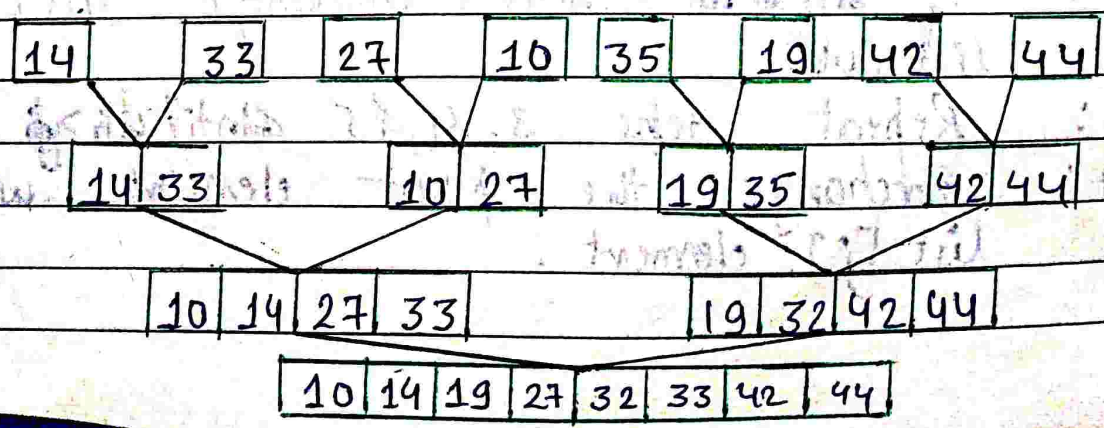
Step 2 - Now, divide the two halves into two more equal halves using
 $halfArray = len(array) / 2$



Step-3- Now further divide the last four pair into single - single element



Step-4- Now we will combine the elements from last nodes in the same manner we splitted them, but first we'll compare each array element then combine them into either ascending or descending order.



QUICK SORT

Quick Sort is a fast sorting algorithm used to sort a list of elements. It was invented by C.A.R. Hoare. This algorithm works upon the divide & Conquer strategy. In quick sort the partition of the list is performed based on the element called pivot. The pivot element is one of the element in the list.

The list is divided into two portions such that all the elements to the left of pivot are smaller than the pivot and all elements to right of pivot are greater than or equal to pivot.

Algorithm - Step 1: Consider the first element of the list as pivot.

Step 2: Define two variables lb and ub. Set lb and ub to first and last elements of the list.

Step 3: Increment lb until $\text{list}[lb] > \text{pivot}$, then stop.

Step 4: Decrement ub until $\text{list}[ub] < \text{pivot}$, then stop.

Step 5: if $lb > ub$ then exchange $\text{list}[lb]$ and $\text{list}[ub]$

Step 6: Repeat steps 3, 4, & 5 until $lb \geq ub$

Step 7: Exchange the pivot element with $\text{list}[j]$ element.

```

Pseudo Code - void quicksort (int arr[10], int first, int last)
{
    int pivot, i, j, temp;
    if (first < last) {
        pivot = first;
        i = first;
        j = last;
        while (i < j) {
            while (arr[i] < arr[pivot]) i++ i++;
            while (arr[j] > arr[pivot] and j > i) j--;
            if (i < j) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        temp = arr[pivot];
        arr[pivot] = arr[j];
        arr[j] = temp;
        quicksort (arr, first, j-1);
        quicksort (arr, j+1, last);
    }
}

```

e.g- To understand Quick sort let us take an Unsorted array $A = \{14, 33, 27, 10, 35, 19, 42, 44\}$ and try to solve it using Quick sort.

Step-1 - lb को तब तक बढ़ाओ जब तक $lb > P$ न मिले
फिर

Step-2 - ub को तब तक घटाओ जब तक $ub < P$ न मिले

Step-3 - अब, जब lb, ub को Cross कर देगा तब $P \geq ub$ करेगा

14	33	27	10	35	19	42	44
lb/p							ub

Step-1.	14	33	27	10	35	19	42	44
	P	lb						ub

	0	1	2	3	4	5	6	7
Step-2.	14	33	27	10	35	19	42	44
	P	lb		ub				

	0	1	2	3	4	5	6	7
Step-3.	14	10	27	33	35	19	42	44
	P	lb		ub				

Step-4.	14	10	27	33	35	19	42	44
	P	ub	lb					

Step-5.	10	14	27	33	35	19	42	44
---------	----	----	----	----	----	----	----	----

Sorted

Unsorted.

SELECTION SORT

Now take the unsorted array -

27	33	35	19	42	44
p/lb				ub	

step-1.

27	33	35	19	42	44
p	lb		ub		

step-2.

27	19	35	33	42	44
p	lb		ub		

step-3.

27	19	35	33	42	44
p	ub	lb			

step-4.

19	27	35	33	42	44
sorted		unsorted			

Now take the unsorted array

35	33	42	44
p/lb			ub

step-1.

35	33	42	44
p	ub	lb	

step-2

33	35	42	44
sorted.			

Best Case - When Pivot is ~~1st (lb)~~
array $[len(array) // 2]$
 $O(n \log n)$

Average case - When array elements are in jumbled order i.e. not properly in increasing or decreasing
 $O(n \log n)$

Worst Case - Pivot is either greater element or smallest element
 $O(n^2)$

SELECTION SORT

Selection sort is a divide and conquer based technique in which we find the minimum element in every iteration and place it in the array beginning from the first index. It is a comparison based algorithm and an inplace sorting technique. It is an unstable sorting algorithm, i.e. doesn't preserve the order of duplicate elements. It has a time complexity of $O(n^2)$.

Algorithm - Step 1: Find the smallest element in the array and swap it with the first element.

Step 2: The element left for sorting are $n-1$ so far. Find the smallest element in the array from index 1 to $n-1$ and swap it with element at index 1.

Step 3: Continue this process for all the elements in the array. Until we get a sorted array.

Pseudocode -

```

list: array of items
n: size of list
for i = 1 to n-1
    min = i
    for j = i+1 to n

```

```

if list[j] < list[min] then
    min = j;
end if
end for
if indexMin != i then
    swap list[min] and list[i]
end if
end for
end procedure.

```

eg - $A = \{ 40, 50, 10, 90, 30 \}$

step 1 -

40	50	10	90	30
i	j			

step 2 -

40	50	10	90	30
i		j		

step 3 -

10	50	40	90	30
	i	j		

step 4 -

10	50	40	90	30
	i		j	

step 5 -

10	50	40	90	30
	i		j	

step 6 -

10	30	40	90	50
	i	j		

step - 7 -

10	30	40	90	50
			i	j

step - 8 -

10	30	40	50	90
----	----	----	----	----

STRASSEN'S MATRIX MULTIPLICATION

The Strassen's algorithm is a divide & conquer based recursive method for matrix multiplication in which each recursive step is dividing the matrix into four sub-matrices of dimension $n/2 \times n/2$. The Strassen's matrix multiplication approach uses just seven recursive multiplications of $n/2 \times n/2$ matrix resulting in an $O(n^{2.81})$ running time complexity.

Let us take an example -

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \times \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} I & J \\ K & L \end{bmatrix}$$

Using Strassen's algorithm -

$$M_1 = (A + D) \times (E + H)$$

$$M_2 = (C + A) \times E$$

$$M_3 = A \times (F - H)$$

$$M_4 = D \times (G - E)$$

$$M_5 = (A + B) \times H$$

$$M_6 = (C - A) \times (E + F)$$

$$M_7 = (B - D) \times (G + H)$$

Then

$$I = (M_1 + M_4) - (M_5 - M_7)$$

$$J = M_3 + M_5$$

$$K = M_2 + M_4$$

$$L = (M_1 + M_3) - (M_2 + M_6)$$

UNIT - 02

GREEDY METHOD

The Greedy method is the simplest and straightforward approach among all the algorithmic approaches to solve a problem. In this approach the decision is taken on the basis of current available information without worrying about the effect of current decision in future.

Greedy algorithm build a solution part by part choosing the next part in such a way, that it gives an immediate benefit. This approach never reconsiders the choices taken previously. This approach is mainly used to solve optimization problems. Greedy method is easy to implement and quite efficient in most of the cases.

In many problems, it doesn't produce an optimal solution though it gives an approximate {near solution} in a reasonable time.

key Terminologies :-

- ① Objective function - This is the function whose output we require to maximize or minimize for a given set of inputs.
e.g. - $f(m)$ = minimum coins required to make a change of 'm' rupees.
- ② Candidate set - It is the collection of items that the greedy algorithm takes as input. It can be a set of numbers, nodes of a graph, characters of a string, etc. The solution for the global optimization is a subset of the candidate set.
- ③ Objective function Optimal solution - It is the solution which is the most favorable solution and best solution in the subset.
- ④ feasible solution - It is the solution which satisfies the given criteria to solve or being the solution of a problem optimally solved.

Advantages of Greedy -

- Easy to implement.
- Require much less computing resources.
- Much faster to execute.
- Used to solve optimization problems.

Disadvantages of Greedy -

- Greedy problems not always reach the global optimum solution.
- Even when the global optimum solution is not reached, most of the times the reached sub-optimal solution is a very good solution.

Knapsack Problem - In this problem you are given -

- A knapsack with limited capacity
- Few items each have some weight and value

The problem states that which item should be selected, so that can be put into the knapsack and

- The value or profit obtained by putting the item is maximum
- The weight inside the knapsack be minimum and shouldn't exceed the weight limit of the knapsack.

Fractional Knapsack Problem

items	1	2	3	4	5
Weight	5	10	15	22	25
Profit	30	40	45	77	90
Capacity	60 kg				

1. Calculate P/w ration for each item

items	1	2	3	4	5
P/w%	6	4	3	3.5	3.6
Weight	5	10	15	22	25

2. Arrange the items in descending order of their P/w ration.

(item_i)_{desc.} →

1	2	5	4	3
---	---	---	---	---

3. Putting items in the knapsack in above sequence in whole.

item-4	
item-5	} 25 (capacity 45 - 25 = 20)
item-2	} 10 (capacity 55 - 10 = 45)
item-1	} 5 (capacity 60 - 5 = 55)

4. Now, we can't put the whole item 4 as the knapsack capacity will exceed so we'll put a fraction of it.

profit Count = whole profit + ~~remaining knapsack limit~~ ^{profit of last item that can be put} ~~wright of last item that~~

so, total profit Count =

$$\begin{aligned} & (30 + 40 + 90) + \frac{20}{22} \times 77 \\ &= 160 + 10 \times 7 \\ &= 160 + 70 \\ &= 230 \end{aligned}$$

Job Sequencing problem - This problem states that "we are given with some jobs along with their profit and deadlines. The profit of Job will be given only when the job is done without exceeding time limit i.e. deadline and only one job can be processed in a unit time as we have a single processor with no multitasking."

Trick - "Deadline के Corresponding जितना हो सके उस Job को उतना देर से पूरना करना है, तब जाके maximum profit मिलेगा।"

Jobs	J_1	J_2	J_3	J_4	J_5	J_6
Deadlines	5	3	3	2	4	2
Profits	200	180	190	300	120	100

1. Sort all the jobs in descending order of their profit.

Jobs	J_4	J_1	J_3	J_2	J_5	J_6
Deadlines	2	5	3	3	4	2
Profits	300	200	190	180	120	100

2. make a optimal job schedule / chart

0	1	2	3	4	5
J_2	J_4	J_3	J_5	J_1	

3. Calculate maximum earned profit -

$$\begin{aligned}
 \text{max profit} &= \text{Sum of profits of all optimally scheduled jobs} \\
 &= P_2 + P_4 + P_3 + P_5 + P_1 \\
 &= 180 + 300 + 190 + 120 + 200 \\
 &= 990
 \end{aligned}$$

Minimum Spanning Tree - A spanning tree is a subset of an undirected graph that has all the vertices connected by minimum number of edges. If all the vertices are connected in the graph then, there exists minimum one spanning tree.

The no. of possible spanning tree from a graph

$$= \boxed{n^{n-2}}$$

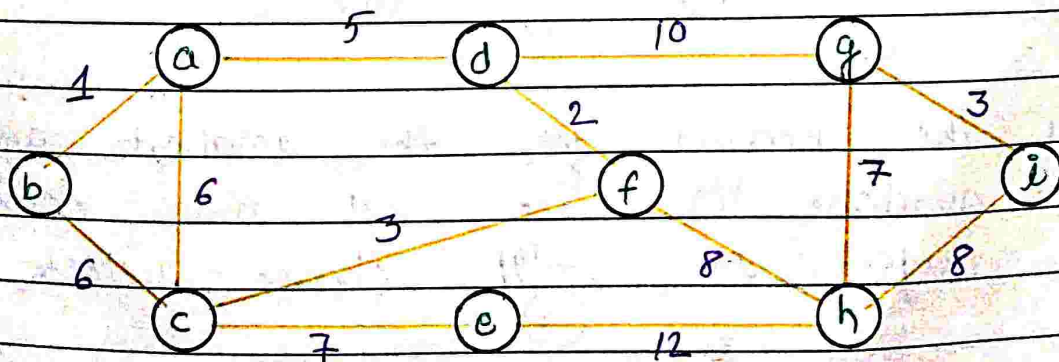
A minimum spanning tree is a subset of a graph that contains edges with minimum cost to cover whole node of the graph.

To derive minimum spanning tree from a graph are've two methods.

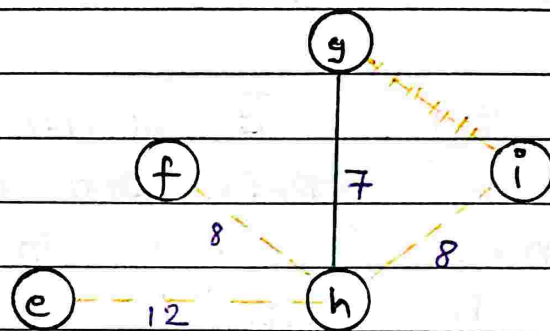
1. Prim's Algorithm - It always start with a single node and it moves through several adjacent nodes, in order to explore all the connected edges along the way.

It starts with an empty spanning tree. The idea is to maintain two sets of vertices. The first set contain the vertices already included in the MST, the other set contains the vertices not yet included. At every step, it considers all edges that connect the two sets and picks the minimum weight edge.

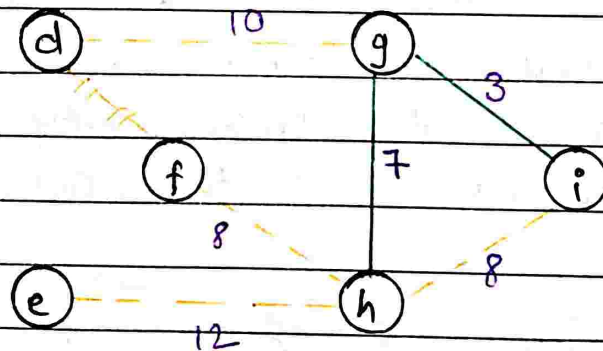
- You can start from any vertex
- first priority is to not make a loop or cycle.



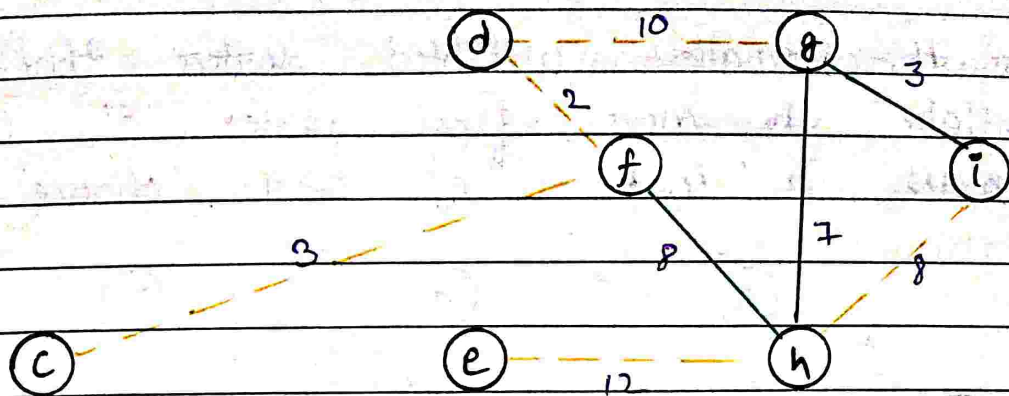
1. Choose a random vertex as starting point from the graph. Let us choose 'h'.
So, nearest vertex from 'h' with minimum edge cost is 'g'. So.



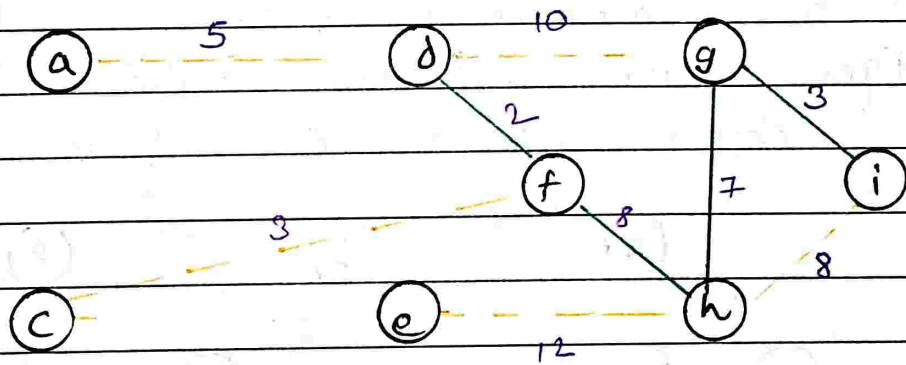
2. Choose the nearest vertex from node 'g' with minimum edge weight that doesn't make a loop. Also consider the edge weight of adjacent edges of nodes of node 'h'.



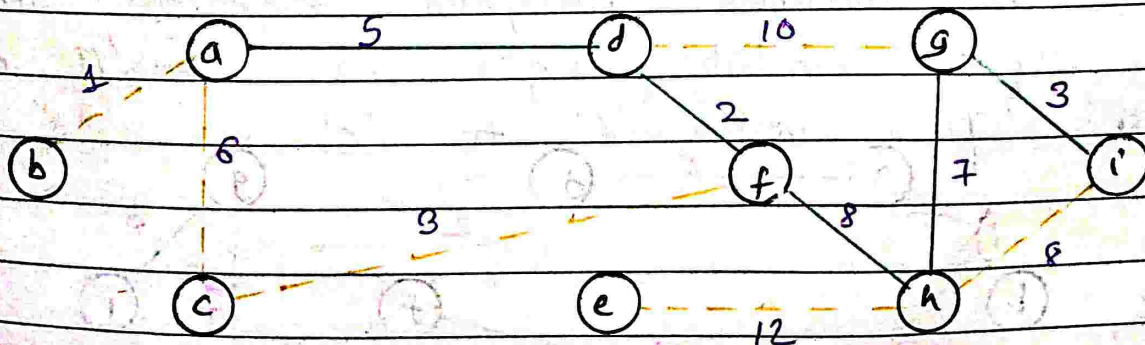
3. Choose the nearest ~~for~~ the smallest weight edge available in the set that doesn't make a loop. So we'll choose vertex 'f'.



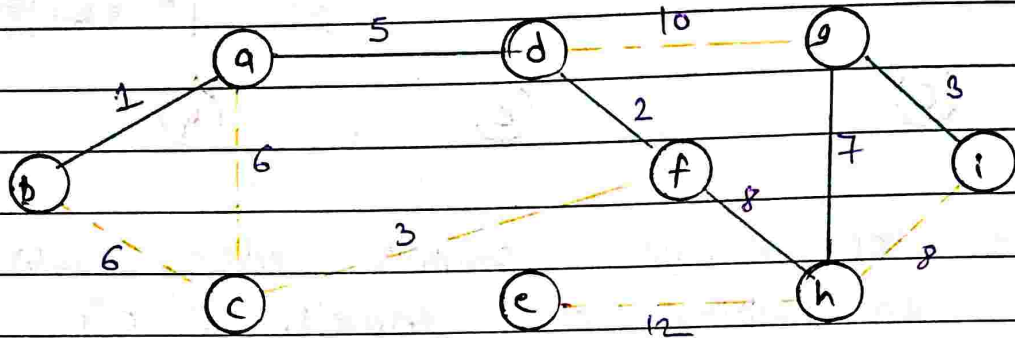
4. Choose the smallest weighted edge from the available edges, so option is '2' as we'll choose vertex 'd'. also it is not making a loop.



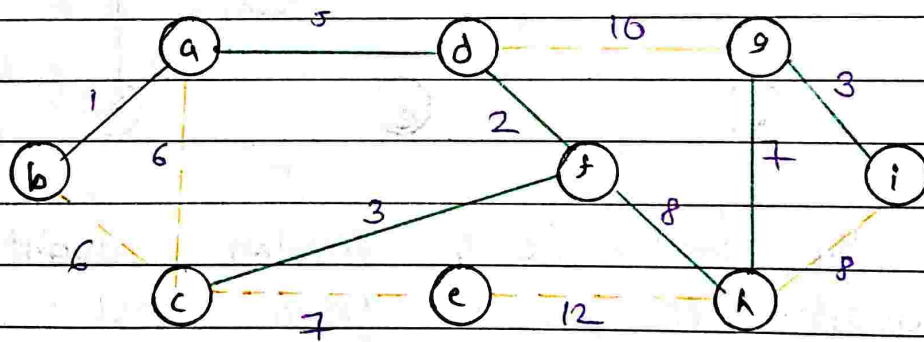
5. Choose the vertex with smallest weight that can be available to move from vertex 'd'. so we'll choose vertex 'a' also it's not making a loop.



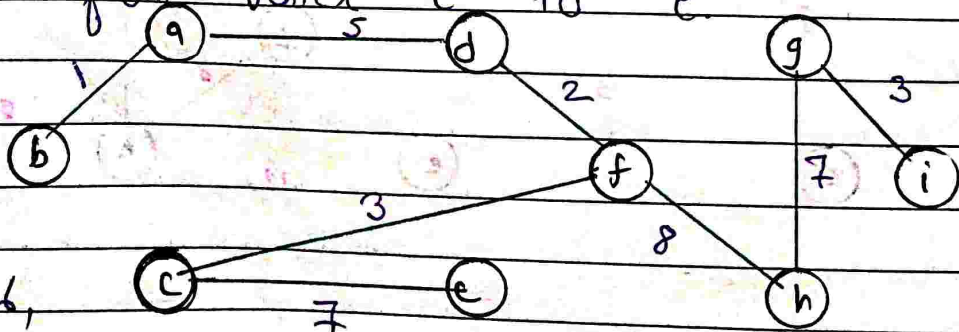
6. Choose the smallest weighted vertex that can be available to move from vertex 'a' which will not make a loop. so we'll choose vertex 'b'.



7. Choose the minimum weighted edge from the available edges that will not make a loop so, we'll choose vertex 'c' from 'f'.



8. Choose the minimum weighted edge to vertex 'e' from available edges so we'll choose path from vertex 'c' to 'e'.



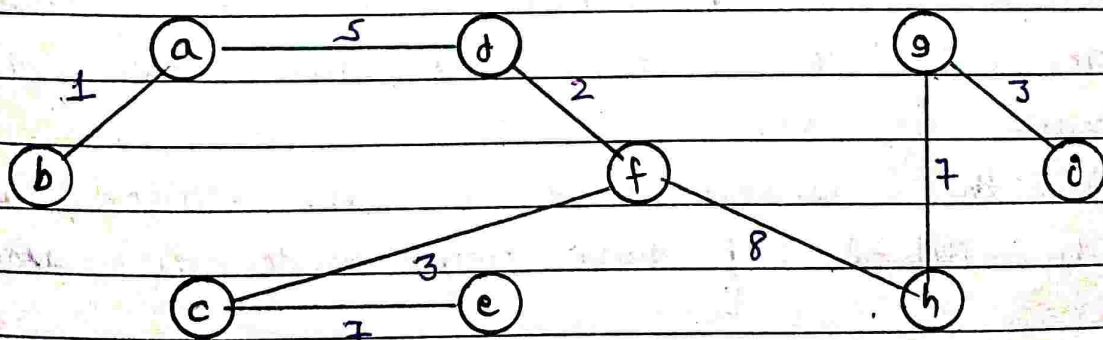
$b \rightarrow a = 1$, $b \rightarrow d = 6$,
 $b \rightarrow f = 4$, $b \rightarrow c = 11$,
 $b \rightarrow e = 18$, $b \rightarrow h = 16$,
 $b \rightarrow g = 28$, $b \rightarrow i = 26$

Minimum Cost - 36

2 Kruskal's Algorithm - It always start with the edge carrying minimum weight. Covers all the nodes in ~~decreasing~~ increasing order of their weight and avoids cycle formation.

Weight Source $\longleftrightarrow$ Destination.

1	a	b	✓
2	d	f	✓
3	c	f	✓
3	g	i	✓
5	a	d	✓
6	a	c	x
6	b	c	x
7	c	e	✓
7	g	h	✓
8	f	h	✓
8	h	i	x
10	d	g	x
12	e	h	x

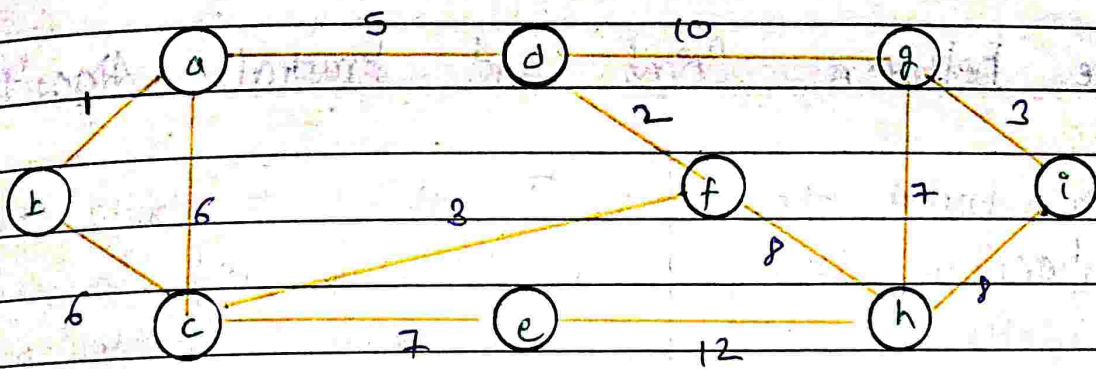


$$b \rightarrow a = 1, \quad b \rightarrow d = 6, \quad b \rightarrow f = 8, \quad b \rightarrow c = 11, \quad b \rightarrow e = 18, \\ b \rightarrow h = 15, \quad b \rightarrow g = 23, \quad b \rightarrow i = 26$$

3. Single shortest path Algorithm - Dijkstra's Algorithm

Dijkstra's Algorithm was introduced by Computer Scientist Edsger W. Dijkstra in 1956. It is a single source shortest path algorithm. It means that it finds the shortest path from a single vertex to all other vertices in a graph. It is a greedy algorithm and works for both directed and undirected graphs but can't work with negatively weighted graphs.

- Algorithm -
- Mark the source node with a current distance of 0 and the rest with infinity.
 - Set the node with the smallest distance as the current node (say C).
 - For each neighbour 'N' (say) of the current node 'C' add the current distance of 'C' with the weight of the edge connecting 'C' - 'N'. If it is smaller than the current distance of 'N', set it as the new current distance of 'N'.
 - Mark the current node 'C' as visited.
 - Go to step-2 if there are any nodes unvisited.

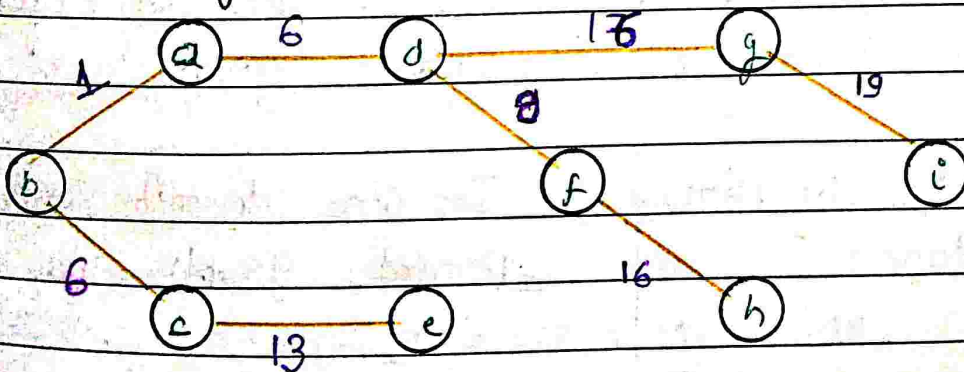


Source

Destination

	a	c	d	e	f	g	h	i
b	①	6	∞	∞	∞	∞	∞	∞
b, a		⑥	6	∞	∞	∞	∞	∞
b, a, c			⑥	13	9	∞	∞	∞
b, a, c, d				13	⑧	16	∞	∞
b, a, c, d, f				⑬		16	16	∞
b, a, c, d, f, e						⑬	16	∞
b, a, c, d, f, e, g							⑬	19
b, a, c, d, f, e, g, h								⑮
b, a, c, d, f, e, g, h, i								

Order of traversing - b, a, c, d, f, e, g, h, i



$b \rightarrow a = 1$, $b \rightarrow c = 6$, $b \rightarrow d = 6$, $b \rightarrow e = 13$, $b \rightarrow f = 8$,
 $b \rightarrow h = 16$, $b \rightarrow g = 16$, $b \rightarrow i = 19$

Difference between Prim's and Kruskal's Algorithm.

Prim's

Kruskal's

- | | |
|---|---|
| <ul style="list-style-type: none">• It starts to build the minimum spanning tree from any vertex | <ul style="list-style-type: none">• It starts to build the minimum spanning tree from the vertex carrying minimum weight |
| <ul style="list-style-type: none">• It traverses one node more than one time | <ul style="list-style-type: none">• It traverses one node only one time |
| <ul style="list-style-type: none">• It takes more time than Kruskal's algorithm | <ul style="list-style-type: none">• It takes less time than Prim's algorithm |
| <ul style="list-style-type: none">• Time Complexity : $O(V^2)$
where, V = no. of vertices | <ul style="list-style-type: none">• Time Complexity : $O(V \log V)$
where V = no. of vertices |
| <ul style="list-style-type: none">• It works only on connected graph component | <ul style="list-style-type: none">• It can also work on disconnected graph component |
| <ul style="list-style-type: none">• It is preferred for dense graph | <ul style="list-style-type: none">• It is preferred for sparse graph |
| <ul style="list-style-type: none">• It doesn't work in directed graph | <ul style="list-style-type: none">• It also doesn't work in directed graph. |

UNIT-03

DYNAMIC PROGRAMMING

Dynamic Programming is a technique that breaks the problem into sub-problems, and saves the result for future purposes so that we do not need to compute the result again. The main use of dynamic programming is to solve optimization problems. Here, optimization problem means that we are trying to find out the minimum or the maximum solution of a problem. The dynamic programming guarantees ~~the~~ to find the optimal solution of the problem if the solution exists.

The definition of dynamic programming says that "It is a technique for solving a complex problem by first breaking it into a collection of simpler subproblems, solving each subproblem just once, and then storing their solutions to avoid repetitive computations."

Principle of Optimality - The Bellman's principle of optimality said that - "Whatever the initial state and initial decisions are, the remaining decision must constitute an optimal solution with regard to the state resulting from the first decision."

No. of Possible Binary Search Trees with 'n' nodes $\Rightarrow \frac{(2n)!}{n!(n+1)!}$

Page: Raj
Date: / /

Optimal Binary Search Tree -

Q. Using dynamic programming find the optimal binary search tree solution

key no.	1	2	3	4
keys	10	12	16	21
frequency	4	2	6	3

Sol. formula used $\Rightarrow C[i, j] = C[i, 0] + C[1, j] + \dots + C[i, j-1] + C[j, j]$

And, when, $j-i=1 \Rightarrow C[i, j] = \text{frequency given}$
 $i=j$ or $j-i=0 \Rightarrow C[i, j] = 0$

$i \backslash j$	0	1	2	3	4
0	0	4	8 ¹	20 ³	26 ³
1		0	2	10 ³	16 ³
2			0	6	12 ³
3				0	3
4					0

Now,

$$C[0, 2] \Rightarrow C[0, 0] + C[1, 2] + 6 = 0 + 2 + 6 = 8^1$$

$$C[0, 1] + C[2, 2] + 6 = 4 + 0 + 6 = 10$$

$$C[1, 3] \Rightarrow C[1, 0] + C[1, 3] + 8 = \text{Not known}$$

$$C[1, 1] + C[2, 3] + 8 = 0 + 6 + 8 = 14$$

$$C[1, 2] + C[3, 3] + 8 = 2 + 0 + 8 = 10^3$$

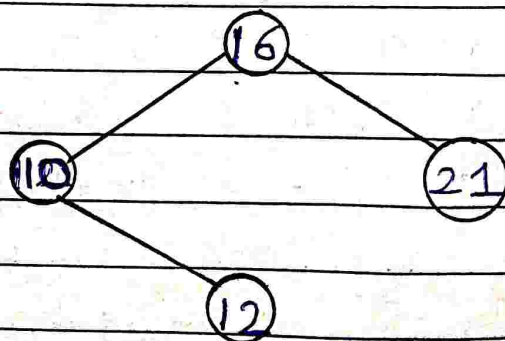
$$\begin{aligned}
 C[2,4] &\Rightarrow C[2,0] + C[1,4] = \text{Not known} \\
 C[2,1] + C[2,4] &= \text{Not known} \\
 C[2,2] + C[3,4] &= 0 + 4 + 9 = 13 \\
 C[2,3] + C[4,4] &= 6 + 0 + 9 = 15
 \end{aligned}$$

$$\begin{aligned}
 C[0,3] &\Rightarrow C[0,0] + C[1,3] = 0 + 10 + 12 = 22 \\
 C[0,1] + C[2,3] &= 4 + 6 + 12 = 22 \\
 C[0,2] + C[3,3] &= 8 + 0 + 12 = 20
 \end{aligned}$$

$$\begin{aligned}
 C[1,4] &\Rightarrow C[0,4] + C[1,4] = \text{Not known} \\
 C[1,1] + C[2,4] &= 0 + 12 + 11 = 23 \\
 C[1,2] + C[3,4] &= 2 + 3 + 11 = 16 \\
 C[1,3] + C[4,4] &= 10 + 0 + 11 = 21
 \end{aligned}$$

$$\begin{aligned}
 C[0,4] &\Rightarrow C[0,0] + C[1,4] = 0 + 16 + 15 = 31 \\
 C[0,1] + C[2,4] &= 4 + 12 + 15 = 31 \\
 C[0,2] + C[3,4] &= 8 + 3 + 15 = 26 \\
 C[0,3] + C[4,4] &= 20 + 0 + 15 = 35
 \end{aligned}$$

The last value inserted to the table was of 3rd key no.
so the root node will be key no. 03.



0/1 Knapsack Problem -

Q. Solve the dynamic knapsack problem for the given data if the capacity of the knapsack is eight.

Weight	3	4	6	5
Profit	2	3	1	4

Sol.

Step-1. Assign item numbers

item	1	2	3	4
Weight	3	4	6	5
Profit	2	3	1	4

Step-2 make a matrix table, containing limit of Column equal to knapsack Capacity and row equal to number of items.

P_i	W_i	$i \backslash j$	0	1	2	3	4	5	6	7	8	
		0	0	0	0	0	0	0	0	0	0	
2	3	1	0	0	0	2	2	2	2	2	2	✓
3	4	2	0	0	0	2	3	3	3	5	5	
4	5	3	0	0	0	2	3	4	4	5	6	✓
1	6	4	0	0	0	2	3	4	4	5	6	

Step-3 - mark all the value for 0th row equal to zero and zeroth column equal to zero as no item is in the knapsack so no profit also.

Step-4 - Now use the following formula to fill rest of the values

$$T(i, j) = \max \{ T(i-1, j), (Value)_i, T(i-1, j-(weight)_i) \}$$

* If The item no. is not there or if a value comes negative

$$\begin{aligned} \text{e.g. } T(2, 1) &= \max \{ T(1, 1), (Value)_2, T(1, 1-(weight)_2) \} \\ &= \max \{ 0, 3, T(1, 1-4) \} \\ &= \max \{ 0, 3, T(1, -3) \} \\ &\quad \rightarrow \text{negative} \end{aligned}$$

$$T(2, 1) = 0$$

Now we'll consider the above rules and situation and proceed to fill the table matrix.

Step-5 - Now we'll check the highest value in the last row and further we'll check if it is present in the above rows or not. If it is present in above rows we'll stop till the last row that contains that value.

in just second row so, we'll consider item 4th in the knapsack and doesn't put the 3rd item in the knapsack.

$$\text{knapsack} = \{x_4\}$$

step-6 - Now remaining weight of knapsack

$$\begin{aligned}\text{capacity remaining} &= 8 - (\text{weight})_4 \\ &= 8 - 5 \\ &= 3\end{aligned}$$

step-7 - Now if we have an item of weight 3 or less than three that can give maximum profit then we'll include that item.

So here we'll include item 1st in the knapsack

$$\text{knapsack} = \{x_4, x_1\}$$

step-8 - Calculating maximum profit

$$\begin{aligned}\text{maximum profit} &= (\text{profit})_4 + (\text{profit})_1 \\ &= 4 + 2 \\ &= 6\end{aligned}$$

Q. Explain the importance of knapsack algorithm in daily life

Ans $\Rightarrow$ Knapsack problem appears in real-world decision making process in a wide variety of fields, such as finding the least wasteful way to cut raw materials, selection of investments and portfolios, etc. One early application of knapsack algorithm was in the construction of tests in which the test-takers have a choice as to which question they answer.

For example, an exam contains twelve questions each worth ten (10) marks, the test-taker need only 10 answers to achieve maximum possible marks of 100. However, on tests with a heterogeneous distribution of marks, it is more difficult to make choices. Furman and Weiss proposed a system in which students are given a heterogeneous test with a total of 125 possible points. The students are asked to answer all the questions to the best of their abilities of the possible subset of problems whose total point value add up to 100, a knapsack algorithm would determine which subset gives each student the highest possible score.

Applications -

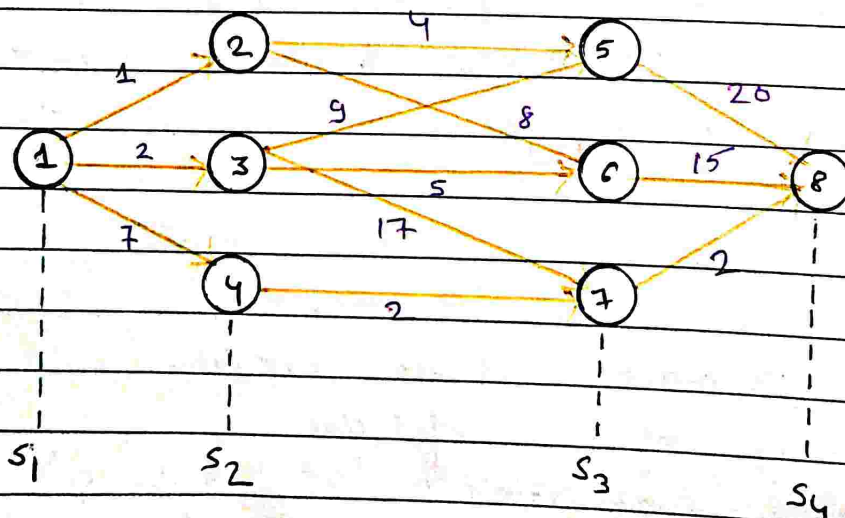
- Home Energy management
- Cognitive radio networks
- Resource management in software
- power allocation management
- 5G mobile edge computing

Multistage Graph - A multistage graph is a directed unweighted graph in which the nodes can be divided into a set of stages such that all edges are from a stage to next stage only i.e. There is no edges between vertices of same stage.

The vertices of multistage graph is divided into 'n' number of disjoint subsets, $S = \{s_1, s_2, \dots, s_n\}$ where s_1 is the source and s_n is the destination.

The Cardinality of s_1 and s_n are equal to 1. We are given a multistage graph a source and a destination, we need to find shortest path from source to destination. By convention, we consider source at stage 1 and destination at last stage s_n {say}.

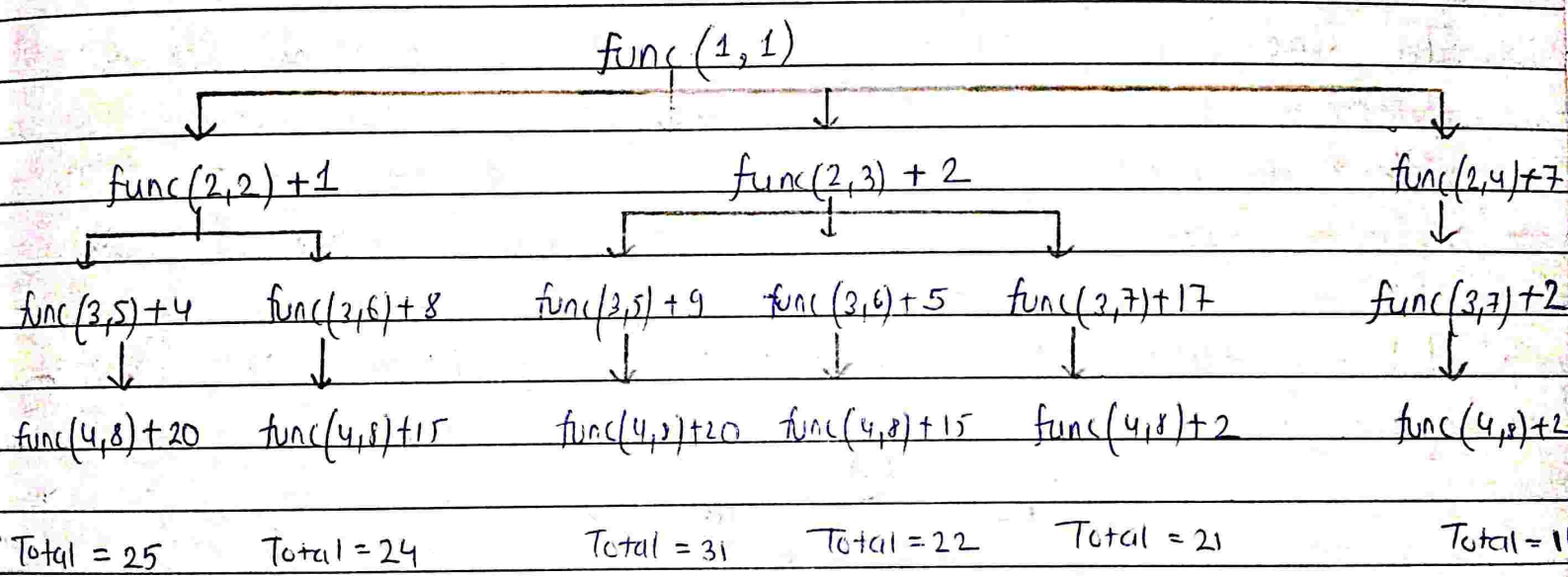
Consider the following example



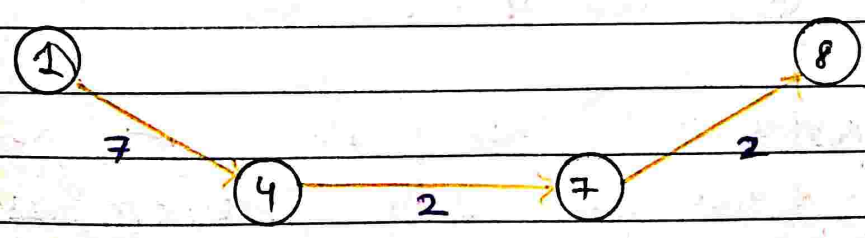
Recursive eqⁿ -

$$\text{func}(s_i, v_j) = \begin{cases} \text{func}(s_{i+1}, k) + C(v_j, k) \\ \text{Terminate at } s_{(n-1)} \end{cases}$$

$\text{func}(s_i, v_j)$: stage s_i , vertex at that stage v_j
 $\text{func}(s_{i+1}, k)$: next stage s_{i+1} , vertex at next stage k
 $C(v_j, k)$: cost of reaching k from vertex v_j
 Terminate at $s_{(n-1)}$: last second stage



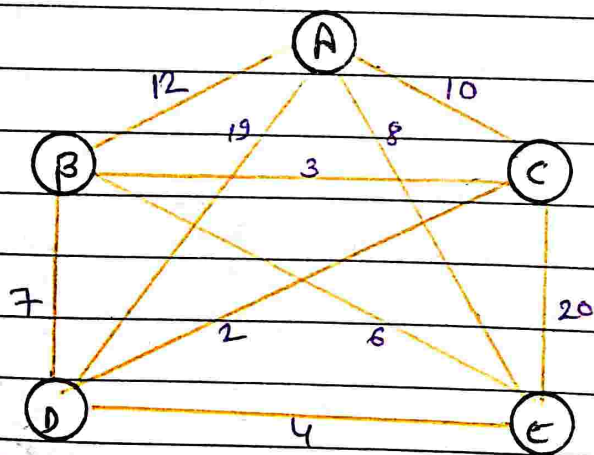
So, the the minimum cost to reach from node-1 {source}, to node -8 {destination} is 11 by following the path below -



Travelling Salesman Problem - The travelling salesman problem (TSP)

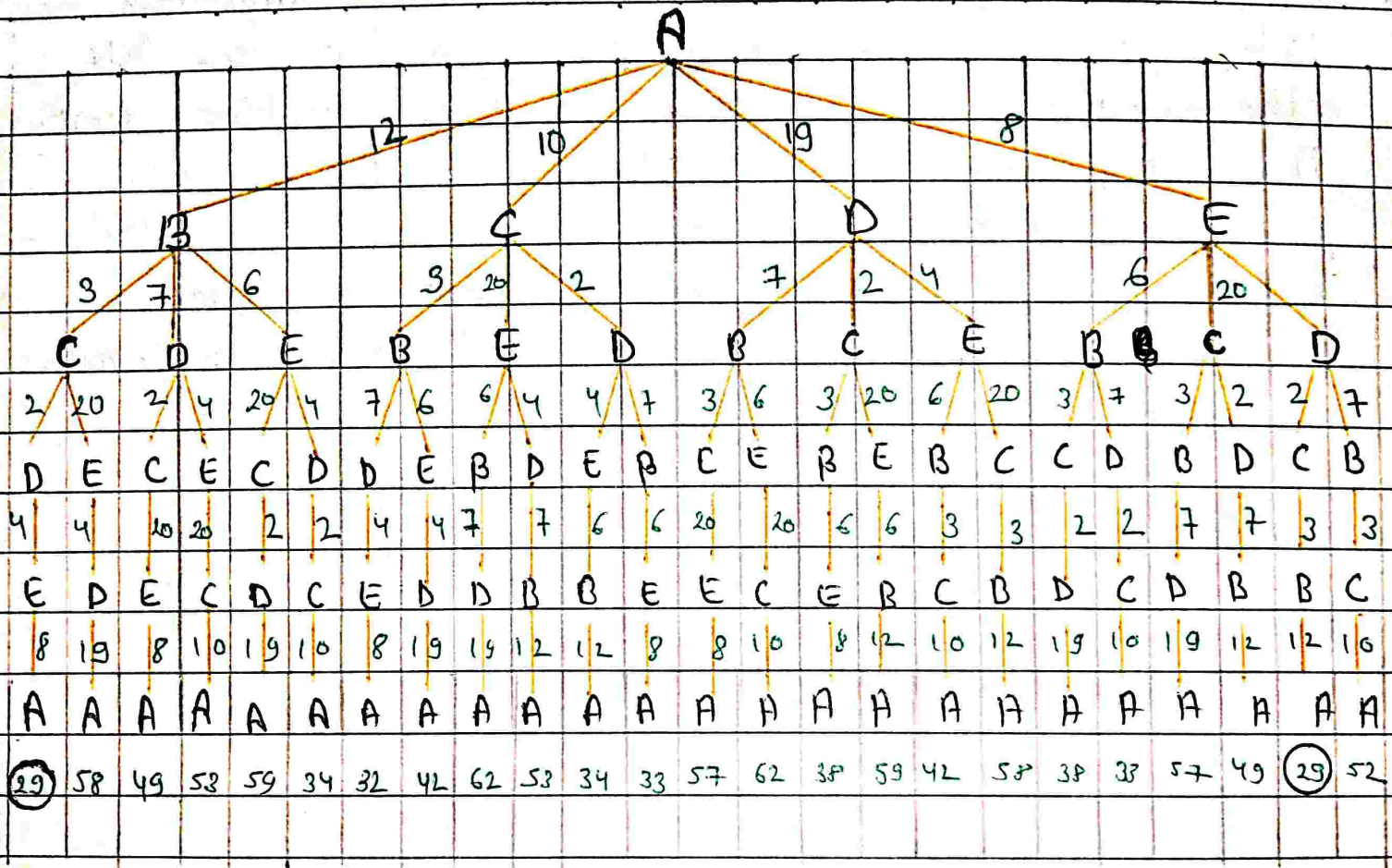
is a very well known problem in theoretical computer science and operations research. The standard version of TSP is a hard problem to solve and belongs to NP-Hard class.

In the TSP, given a set of cities and the distance between each pair of cities, a salesman needs to choose the shortest path to visit every city exactly once and return to the city from where he started. Let's take an example to understand the problem.



Here the nodes represent cities and the weight on each edge denotes the distance between one city to another. Let's assume that a salesman starts his journey from 'A'. According to the TSP, the salesman needs to visit all the cities at exactly once and get back to the city 'A' by choosing the shortest path.

Note - No. of leaf nodes in the tree will be $(n-1)!$
 where n is no. of Cities.



So, the smallest path that can cover all the cities and the salesman come back to his/her city again is -

$A \rightarrow E \rightarrow D \rightarrow C \rightarrow B \rightarrow A$

Or

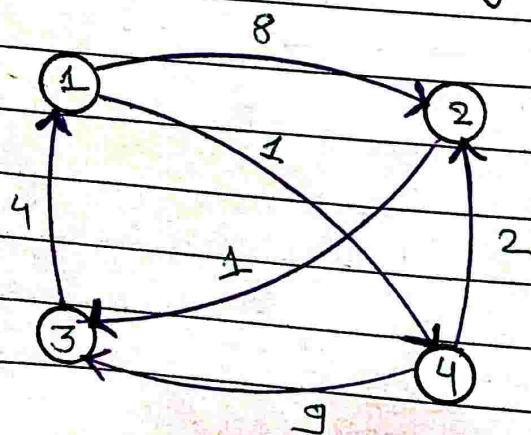
$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$

Floyd Warshall Algorithm {All pair shortest path}

Floyd Warshall Algorithm is an algorithm for finding the shortest path between all the pairs of vertices in a weighted graph. This algorithm works for both the directed and undirected weighted graphs. But it doesn't work with graphs having negative cycles. This algorithm follows the dynamic programming approach to find the shortest path.

Algorithm - $n = \text{no. of vertices}$
 $A = \text{matrix of dimension } n \times n$
for $k = 1$ to n
 for $i = 1$ to n
 for $j = 1$ to n
 $A^k[i, j] = \min \{A^{k-1}[i, j], A^{k-1}[i, k] + A^{k-1}[k, j]\}$

Q. Find the shortest distance between each node of the following graph using Floyd warshall algorithm.



87.

$D^0 =$

	1	2	3	4
1	0	8	∞	1
2	∞	0	1	∞
3	4	∞	0	∞
4	∞	2	9	0

$D^1 =$

{include node-1}

	1	2	3	4
1	0	8	∞	1
2	∞	0	1	∞
3	4	12	0	5
4	∞	2	9	0

$D^2 =$

{include node-1,2}

	1	2	3	4
1	0	8	9	1
2	∞	0	1	∞
3	4	12	0	5
4	∞	2	3	0

$D^3 =$

{include node-1,2,3}

	1	2	3	4
1	0	8	9	1
2	5	0	1	6
3	4	12	0	5
4	7	2	3	0

$D^4 =$

{include node-1,2,3,4}

	1	2	3	4
1	0	3	4	1
2	5	0	1	6
3	4	7	0	5
4	7	2	3	0

This is the required distances between each node in form of matrix.

UNIT - 04

BRANCH & BOUND

Branch and Bounds are basically an algorithm that is used to get the correct method of the solution for the use of the discrete, combination and general mathematical problems that need to be optimized.

This algorithm basically explores the amount of entire space for the search of the possible solutions and it then provides an optimal solution. If the problem that is given is discrete in a manner for the optimization problem then the branch and bound is better choice.

It actually looks for the best solution

Advantages -

- Time complexity is less than other algorithms.

- In case, the given problem is not large and branching of nodes is possible in a reasonable time, then it finds an approximate optimal solution.

Disadvantages -

- mostly time consuming, depends on number of nodes and particular problem.
- Process of parallelization is too difficult in branch and bound.

0/1 knapsack Using Branch and Bound -

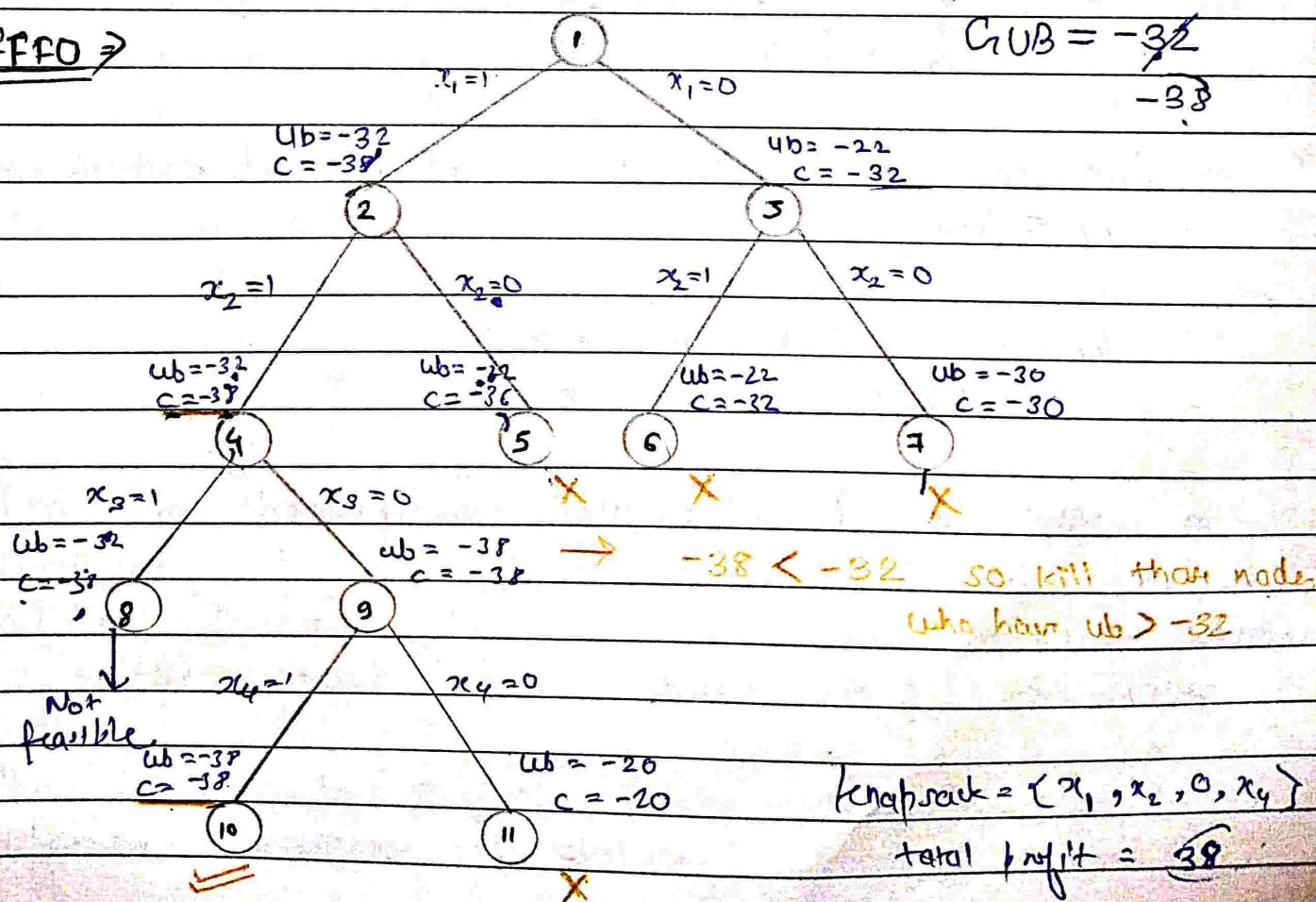
There are three methods to solve 0/1 knapsack using branch and bound -

- FIFO {medium}
- LIFO {lengthy}
- LC {easy}

Q. Solve the Branch & Bound knapsack problem for the given data using FIFO & LC method

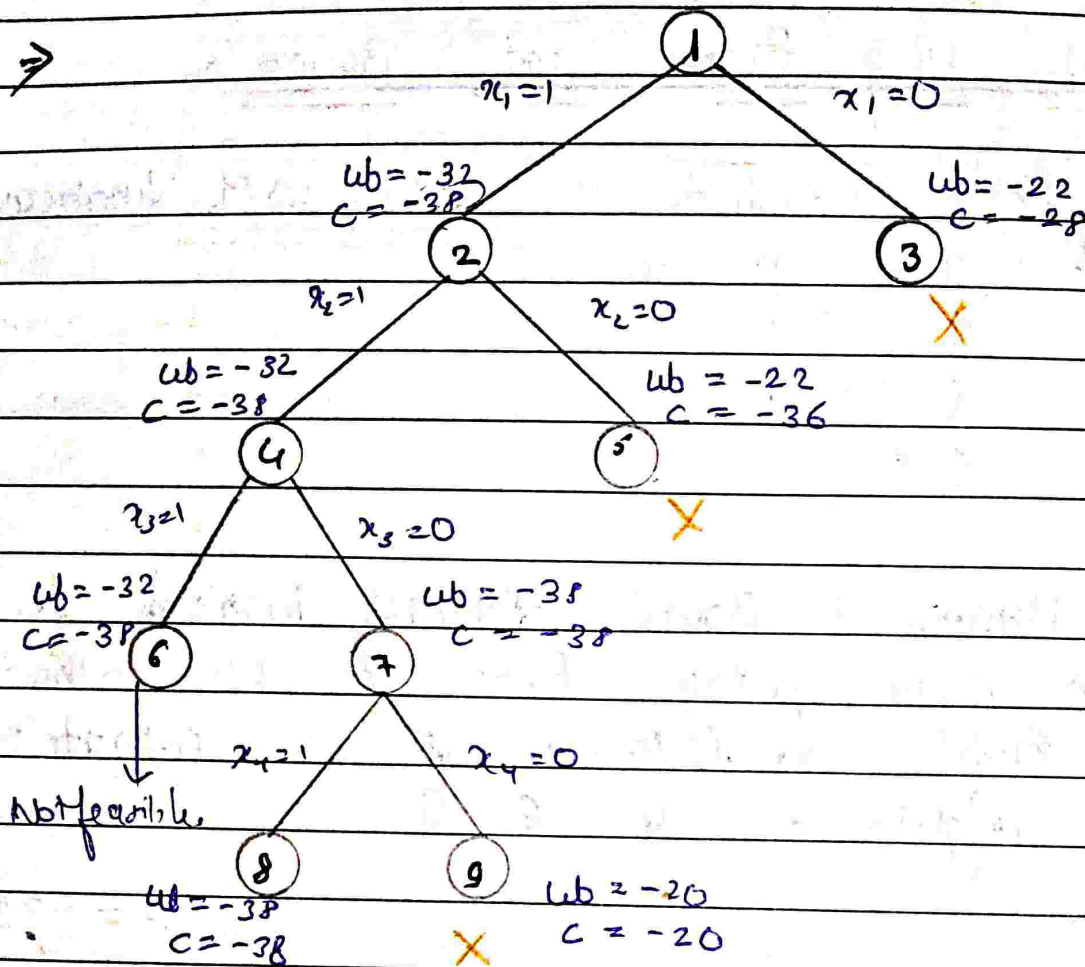
	Profit	10	0	10	12	18	
weight	2	4	6	9			Capacity = 15

FIFO →



LC $\Rightarrow$

GUB = ~~-38~~
-38



* We have to proceed those nodes whose $ub < GUB$ or $ub = GUB$.

knapsack = $\{x_1, x_2, 0, x_4\}$
total profit = 38

or $|GUB| \leq |ub|$ then proceed

BACKTRACKING

Page: _____

Raj

Date: ____/____/____

Backtracking is an algorithmic technique for solving problems recursively by trying to build a solution incrementally, one piece at a time, removing those solutions that failed to satisfy the constraints of the problem at any point in time.

Backtracking can also be said as an improvement to the Brute force method.

So, basically the idea is that it searches for a solution to a problem among all available options.

Initially we start the backtracking from one possible option and if the problem is solved with that selected option then we return the solution else we backtrack and select another option from the remaining available options.

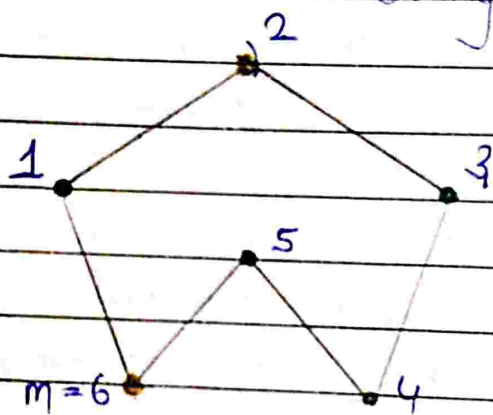
There also might be a case, where none of the options selected will give you a solution.

Graph Colouring Problem -

Given an undirected graph and a number ' m ', determine if the graph can be colored with at most ' m ' colors such that no two adjacent vertices of the graph are coloured with the same color.

The idea is to assign colors one by one to different vertices, starting from vertex 0.

Before assigning a Color, check for safety by considering already assigned colors to the adjacent vertices. i.e. check if the adjacent vertices have the same color or not. If there is any color assignment that does not violate the conditions, mark the color assignment as part of the solution. If no. color assignment is possible then backtracking and return false.



- Algorithm -**
- Create a recursive function ~~that~~ that takes the graph, current index, number of vertices, and output colour array.
 - If the current index is equal to the number of vertices. Print the colour configuration in the output array.
 - Assign a colour to a vertex (1 to m)
 - For every assigned color, check if the configuration is safe
 - Recursively call the function with the next index and number of vertices.
 - If any recursive function returns true, break its loop and return true else return false

N-Queen Problem -

Algorithm - 1. Start in the leftmost column

2. If, all queens are placed correctly
return (True)

3. Else, Try all rows in the current location
Do the following for every tried row.

a) If the queen can be placed safely in this row then mark this [row, column] as a part of solution and recursively check, if placing the queen here, can lead a solution

b) If placing the queen in [row, column] leads to a solution then
return (True)

c) If placing the queen doesn't lead to a solution then unmark this [row, column] and backtrack and go to step (a) to try other rows

4. If all rows have tried and nothing works
return false to trigger backtracking.

Backtracking

- It solves the decision problem
- Some bad choices could be made
- It uses DFS
- The state space tree is searched until the solution of the problem is obtained
- In backtracking, if the solution is not satisfiable, then we backtrack for other possible option
- It is more efficient

Branch & Bound

- It solves the optimization problem
- It provides the better result, no chance of bad choice
- It can use both DFS and BFS
- The state space tree needs to be searched completely as the optimum solⁿ can be present anywhere in tree
- In branch & bound, we calculate bounding values, according to which either we stop or extend
- It is less efficient

COMPLEXITY CLASSES

Page: _____

Date: ____/____/____

P-Class - It is a collection of problems that can be solved by a deterministic machine in polynomial time. And the solution of the problems of this class is easy to find.

- e.g. -
- Calculating HCF
 - decision problems
 - finding a maximum matching

NP-Class - It is a collection of problems that can be solved by a non-deterministic machine in polynomial time. This class contains problems that are verifiable, but may or may not be solvable in polynomial time.

- e.g. -
- Graph Colouring
 - hamilton path problem
 - satisfiability

The solution of NP-class problems is hard to find because they are being solved by a non-deterministic machine.

e.g. - Suppose there are 1000 employees in a company with unique employee IDs. Assume that there are 200 rooms available for them to work. A selection of 200 employees must be paired together, but the CEO of the company has the data of some employees who don't want to work together for some personal reasons. This is an example of NP problem.

NP-hard - It is a collection of problems whose least hard problem is as hard as the most hardest problem of NP-class. These problems are verifiable but very much hard or impracticable to solve in polynomial time. If a solution of an NP-hard problem is given but it is still hard because it will take very long time to check the result whether it is correct or not.

eg. -

- Halting problem
- No Hamilton Cycle

NP-complete / Completeness - It is a collection of problems which belong to both NP-class and NP-hard class. And there is no polynomial time for such problems.

If anyone is able to solve this problem in a given polynomial time then he/she can solve any NP-class problem.

eg. -

- Hamilton Cycle
- Satisfiability
- Decision version 0/1 knapsack

A non-deterministic machine can solve these problems in polynomial time.

NP-Hard

↓
unsolvable in
Polynomial time
but can be
verified

NP

Complete

Can be
verified
in polyno-
mial
time

NP

↓
solvable by
non-deterministic
machine in
Polynomial
time

P

↓
solvable by
deterministic
machine in
Polynomial
time